DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843-01 - SPECIAL TOPIC IN DATA ENGINEERING

TECHNICAL REPORT

TITLE : IMAGE CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORK

NAME	MATRIC NO
NEO LI XIN	A23CS0253
RICHARD YONATHAN JULIO CLAY HENG	X25EC3019
JEANETTE HAUW CHANDRA	X25EC3020

PREPARED FOR:

DR. ARYATI BINTI BAKRI

1.0 Introduction

Artificial Intelligence (AI) has brought revolutionary changes to the functioning of computers in relation to processing and comprehending visual information. An important application of AI is image classification which allows computers to classify and categorize images without requiring human intervention. Traditional machine learning methods necessitate the extraction of features manually, making image processing tedious and imprecise. Deep learning algorithms like Convolutional Neural Networks (CNNs) have been introduced to solve these problems.

CNNs are neural networks designed for processing image-based datasets that automatically detect features within images. CNNs are extensively applied for computer vision tasks such as face recognition, diagnostic imaging, self-driving cars, and object detection due to high precision and pattern detection capabilities. This paper discusses the effectiveness of CNNs as well as their implementation using Python based on a tutorial.

1.1 What is Image Classification?

Image classification refers to the process of labeling a particular image to one or several classes of predefined labels in computer vision tasks. Image classification can allow computers to understand what is contained in a certain image by recognizing it and placing it into a particular class. An image classification system will classify an image into cats, dogs, cars, or planes.

There are various fields where image classification systems play significant roles, such as in the field of medicine for making diagnoses, in security systems to monitor activities, in facial recognition technology, and in automated quality control.

1.2 What is CNN?

CNN refers to a class of Deep Learning models that are explicitly built for image processing applications. Contrary to the traditional Neural Networks where human engineers must manually engineer image features, CNNs have a unique architecture that allows them to automatically extract features in multiple stages.

The general structure of a CNN involves three major layers:

- Convolution Layer – Responsible for extracting relevant image features like edge detection using kernels.

- Pooling Layer – This layer reduces the dimensionality of the feature map without losing significant information, thus lowering computational complexity and avoiding overfitting.

- Fully Connected Layer – Uses the extracted features to perform image classification and generate output predictions.

By combining these layers, CNNs can learn hierarchical image representations, allowing them to achieve high performance in image recognition and classification tasks.

2.0 Evaluation of CNN

CNN has emerged as one of the most effective deep learning algorithms for image recognition because of their capability to learn visual features from unprocessed images automatically. CNNs eliminate the need for manual feature extraction and can identify complex patterns within images.

2.1 Strengths of CNN

In this section, the strengths of CNN are explored to understand the efficiency and effectiveness of this technology. This analysis highlights the key advantages that make CNNs a widely adopted approach in modern computer vision applications.

1. Automatic Feature Extraction

CNNs extract automatic features, including edges, shapes, texture, and component parts of objects from training images, thus eliminating the need for manual feature selection.

2. Higher Classification Accuracy

Typically, CNNs yield high accuracy when classifying data since they have the ability to learn complex representations of images and perform hierarchical learning.

3. Parameter Sharing

The convolution layer of CNNs utilizes a parameter sharing technique in which the same weight is applied in multiple locations of a single image and different images.

4. Translation Invariant

Due to the application of the convolution and pooling layers, CNNs can detect objects even when they are positioned in a different part of an image.

5. Versatile Application

Applications of CNNs include but are not limited to image recognition, face recognition, medical imaging, driverless vehicles, object detection, among others.

3.0 Hands-on Implementation in Python

1. Importing libraries

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 3.1: Importing Required Python Libraries for CNN Development

The required Python libraries were imported to support the development of the CNN model. TensorFlow and Keras were used for deep learning model construction, NumPy was used for numerical operations, and Matplotlib was used for data visualization.

2. Loading the dataset

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
```

Figure 3.2: Loading the CIFAR-10 Dataset

The CIFAR-10 dataset was loaded using the Keras dataset API. The dataset contains 50,000 training images and 10,000 testing images categorized into ten object classes.

3. Inspecting the dataset

```
X_train.shape

(50000, 32, 32, 3)
```

Figure 3.3: Inspection of X_train dimensions

```
X_test.shape

(10000, 32, 32, 3)
```

Figure 3.4: Inspection of X_test dimensions

```
y_train.shape

(50000, 1)
```

Figure 3.5: Inspection of y_train dimensions

```
y_test.shape

(10000, 1)
```

Figure 3.6: Inspection of y_test dimensions

The dataset dimensions were inspected to verify the number of images and labels. The training images have a shape of (50000, 32, 32, 3), while the training labels have a shape of (50000, 1). The testing dataset consists of 10,000 images and corresponding labels.

4. Defining the class labels

```
classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck
```

Figure 3.7: Definition of CIFAR-10 Class Labels

A list of class names was created to map numerical labels to their corresponding object categories, such as airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

5. Defining the function for visualizing images

```
def plot_sample(X, y, index):
  plt.figure(figsize=(15, 2))
  plt.imshow(X[index])
  plt.xlabel(classes[y[index]])
```

Figure 3.8: Definition of the plot_sample Function for Image Visualization

The plot_sample() function was created to display sample images from the CIFAR-10 dataset together with their corresponding class labels. The function accepts the image dataset (X), label dataset (y), and image index as input parameters. Using Matplotlib, the selected image is displayed while its class label is shown below the image, providing a convenient way to inspect and verify the dataset before model training.

6. Visualizing the sample images

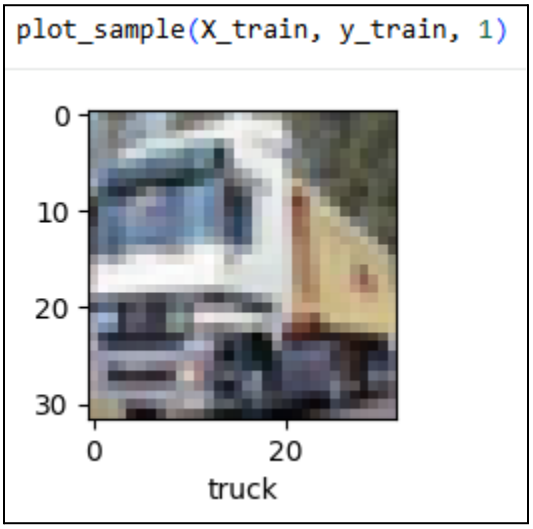

Figure 3.9: Visualization of a Sample Image from the Training Dataset

A sample image from the training dataset was displayed together with its corresponding class label. This step helps verify that the dataset has been loaded correctly and provides a visual understanding of the classification task.

7. Reshaping the labels

```
y_train = y_train.reshape(-1,)
y_train[:5]
```

```
array([6, 9, 9, 4, 1], dtype=uint8)
```

Figure 3.10: Reshaping y_train into One-Dimensional Format

```
y_test = y_test.reshape(-1,)
y_test[:5]
```

```
array([3, 8, 8, 0, 6], dtype=uint8)
```

Figure 3.11: Reshaping y_train into one-dimensional format

The training and testing labels were reshaped from two-dimensional arrays into one-dimensional arrays using the `reshape(-1,)` function. This simplifies label indexing and facilitates model evaluation and visualization.

8. Normalizing the pixels

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Figure 3.12: Normalization of Image Pixel Values

Pixel values were normalized by dividing each value by 255. This scales the pixel range from 0–255 to 0–1, improving training stability and accelerating model convergence.

9. Creating compiling and training the ANN model

```
ann = models.Sequential([
    layers.Flatten(input_shape = (32,32,3)),
    layers.Dense(3000, activation = 'relu'),
    layers.Dense(1000, activation = 'relu'),
    layers.Dense(10, activation = 'softmax')
])

ann.compile(optimizer='SGD',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

ann.fit(X_train, y_train, epochs=5)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: U
  super().__init__(**kwargs)
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 137s 87ms/step - accuracy: 0.3531 - loss: 1.8148
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 129s 83ms/step - accuracy: 0.4250 - loss: 1.6242
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 137s 88ms/step - accuracy: 0.4595 - loss: 1.5401
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 142s 88ms/step - accuracy: 0.4793 - loss: 1.4796
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 135s 86ms/step - accuracy: 0.4968 - loss: 1.4322
<keras.src.callbacks.history.History at 0x7d2bc7f558b0>
```

Figure 3.13: Construction and training of the ANN model

An Artificial Neural Network (ANN) was developed using fully connected layers and trained for five epochs. The training process displays the model accuracy and loss values after each epoch.

10. Evaluating the ANN model through classification report

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification report: \n', classification_report(y_test, y_pred_classes))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 8s 24ms/step
classification report:
              precision    recall  f1-score   support

           0       0.52      0.60      0.56      1000
           1       0.66      0.56      0.61      1000
           2       0.42      0.33      0.37      1000
           3       0.29      0.38      0.33      1000
           4       0.59      0.22      0.32      1000
           5       0.30      0.54      0.39      1000
           6       0.54      0.44      0.48      1000
           7       0.56      0.54      0.55      1000
           8       0.65      0.59      0.62      1000
           9       0.56      0.58      0.57      1000

    accuracy                           0.48     10000
   macro avg       0.51      0.48      0.48     10000
weighted avg       0.51      0.48      0.48     10000
```

Figure 3.14: Classification Report of the ANN Model

The classification report summarizes the ANN model's performance using precision, recall, F1-score, and support metrics for each CIFAR-10 class. These metrics provide a detailed evaluation beyond overall accuracy.

11. Evaluating the ANN model through heatmap of the confusion matrix

```
plt.figure(figsize=(14,7))
sns.heatmap(y_pred, annot = True)
plt.ylabel('Truth')
plt.xlabel('Prediction')
plt.title('Confusion matrix')
plt.show
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

```
/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py
Display all open figures.

Parameters
----------
block : bool, optional
```

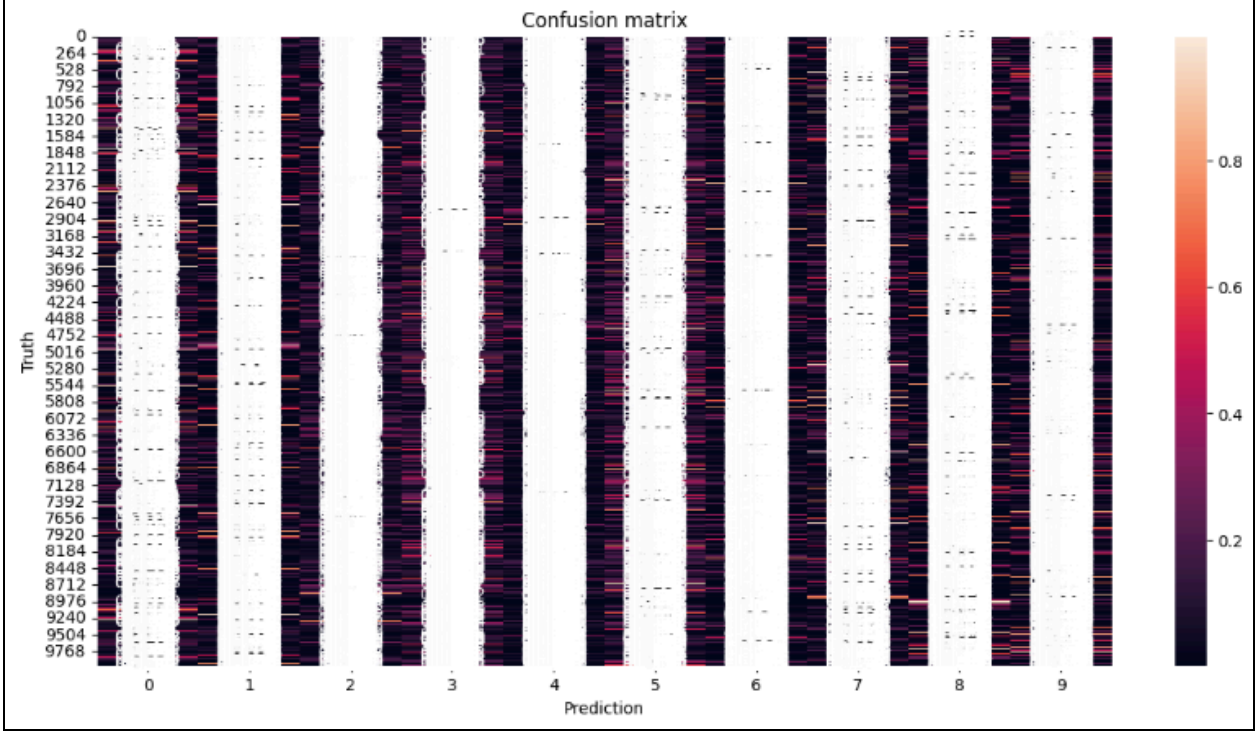

Figure 3.15: Heatmap Visualization of ANN Prediction Results

A heatmap was generated to visualize the prediction outputs of the ANN model. The visualization highlights the distribution of prediction values across different classes and provides insight into the model's classification behavior.

12. Creating, compiling and training the CNN model

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(54, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
cnn.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

```
cnn.fit(X_train, y_train, epochs=10)

Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 63s 39ms/step - accuracy: 0.4810 - loss: 1.4516
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 64s 41ms/step - accuracy: 0.6146 - loss: 1.1016
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 60s 38ms/step - accuracy: 0.6657 - loss: 0.9646
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 61s 39ms/step - accuracy: 0.6940 - loss: 0.8841
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 61s 39ms/step - accuracy: 0.7174 - loss: 0.8157
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 60s 38ms/step - accuracy: 0.7343 - loss: 0.7613
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 61s 39ms/step - accuracy: 0.7531 - loss: 0.7114
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 80s 38ms/step - accuracy: 0.7680 - loss: 0.6666
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 84s 40ms/step - accuracy: 0.7812 - loss: 0.6295
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 65s 41ms/step - accuracy: 0.7895 - loss: 0.5990
<keras.src.callbacks.history.History at 0x7d2b5d482f90>
```

Figure 3.16 - 3.18: Construction and Training of the CNN Model

The CNN architecture consists of convolutional layers for feature extraction, max-pooling layers for dimensionality reduction, and fully connected layers for image classification. This architecture enables automatic learning of image features from the CIFAR-10 dataset. The CNN model was trained for ten epochs using the Adam optimizer. The

training output shows a gradual increase in accuracy and a decrease in loss, indicating successful learning of image features.

13. Evaluating the CNN model

```
cnn.evaluate(X_test, y_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.6995 - loss: 0.9060
[0.9060400128364563, 0.6995000243186951]
```

Figure 3.19: Evaluation Results of the CNN Model on the Test Dataset

The trained CNN model was evaluated using the CIFAR-10 test dataset. The model achieved a test accuracy of approximately 69.95% and a test loss of 0.9060, demonstrating its ability to generalize to previously unseen images.

4.0 Weaknesses of the Current CNN Model

While the original CNN model possesses the ability to classify images, there are some potential limitations associated with the performance of this model. These limitations might impair the process of the model's training, reduce classification accuracy, and adversely affect the generalizability of the CNN model.

1. Overfitting

One of the main drawbacks of the original CNN model is that it is highly prone to overfitting without employing regularization algorithms like Dropout. Consequently, such overfitting causes the model to learn training data too well and fail to generalize learned features onto new, unseen images.

2. Instability during Training

During the training phase, the original CNN model might display certain instability caused by the changing nature of data going through the neural network layers.

3. Poor Monitoring of Generalization Ability

With the absence of a separate dataset to validate the model, there would be difficulties in determining the performance of the model on new data. As such, indications of whether the model experiences overfitting or underfitting will be hard to detect early on.

4. Poor Learning Efficiency

Certain hyperparameters like the learning rate and batch size used during training play a crucial role in the performance of the model. The learning rate could cause issues where an improper learning rate causes fast convergence or oscillations around the optimum. Similarly, using a small batch size could cause poor gradient descent due to noise.

5. Insufficient Training Duration

With limited training epochs, the model would find it difficult to learn certain characteristics of the images contained within the dataset. This results in poor generalization and a less accurate model overall.

5.0 Enhancements to the CNN Model

In order to enhance the CNN architecture in terms of performance and stability, certain improvements had been made. These changes were done to minimize the risk of overfitting, improve the training phase, and make the model better at generalizing.

1. Batch Normalization

Batch normalization layers were used to stabilize the flow of data in the neural network and minimize the chance of either exploding or vanishing gradients, thus increasing the capacity of a neural network to learn. In such a way, the training process becomes smoother and loss functions become more constant.

2. Dropout Layer

A dropout layer was utilized as a method of regularizing the model by randomly turning off a part of neurons while doing backpropagation. Such a regularization method prevents overfitting of the CNN and makes the final output less dependent on specific features.

3. Low Learning Rate (0.0002)

Learning rate of the Adam optimizer was adjusted to 0.0002. The change allowed the network to update the weights slowly, which resulted in fewer fluctuations when plotting the graphs of loss and accuracy for the validation set.

4. Validation Split (0.1)

For validating the results obtained during the training process, we have implemented a validation split of 0.1, which will allocate 10% of the training data towards validation. The use of

validation accuracy and loss measures is crucial for determining the effectiveness of our model's performance on the unseen data.

5. Larger Batch Size (128)

We increased the batch size to 128 images per epoch because this enables a smoother training process by providing more stability in terms of estimating the gradients of each iteration.

6. Increased Number of Epochs (50)

The number of training epochs is increased from 30 to 50 to give our model more time to explore the features of images stored in the database. By using regularizers such as Batch Normalization and Dropout methods, our model could be trained for a long duration without suffering from overfitting.

6.0 Comparative Evaluation

This section compares the baseline CNN and the improved new_CNN on the same dataset and data split to see whether the training changes lead to better performance.

Aspect	Baseline CNN	new_CNN
Performance	Achieves reasonable accuracy and learns useful patterns from the dataset. However, performance plateaus earlier and exhibits a larger gap between training and validation accuracy.	Achieves higher validation and test accuracy with a smaller gap between training and validation accuracy.
Training and Validation	Validation accuracy becomes unstable after several epochs, and validation loss stops decreasing while training accuracy continues to improve.	Validation accuracy and loss curves are smoother and more consistent throughout training.
Test Set Performance	Produces more errors on visually similar classes and shows weaker performance on ambiguous images.	Achieves higher test accuracy and distributes classification errors more evenly across classes.
Effect of Design Changes	Uses a smaller validation set and fewer training epochs,	Uses a larger validation set and extended training epochs

	limiting performance improvement.	with proper monitoring.

Table 6.1: Comparison of Baseline CNN and new_CNN in terms of Training, Validation and Test Set Performance

Table 6.1 compares the baseline CNN and the new_CNN models based on four important characteristics: performance, training and validation performance, performance on the test set, and the impact of architectural changes. Baseline CNN model manages to learn patterns in the data but suffers from a faster performance saturation and a bigger gap between training and validation accuracies, which means overfitting. On the other hand, new_CNN model is capable of demonstrating greater validation and testing accuracy while showing less performance gap between training and validation accuracies.

As for training and validation processes, the baseline model shows unstable validation accuracy with validation loss not improving further at some point despite continuing growth of training accuracy. New_CNN model is characterized by smoother validation graphs and more even learning behavior, which shows that model learns from training examples, not merely remembers them. As to testing performance, the baseline model commits more errors due to ambiguous classes, while new_CNN commits fewer errors and is better at distinguishing between similar classes.

The improvements achieved in the case of new_CNN are a result of the architectural modifications made during the training process. Expanding the number of samples in the validation dataset results in more robust signals and makes any changes observed more stable. Additionally, adding more epochs helps the network to learn better feature detection. If done right, the improvements will yield performance gains without causing overfitting, and a more accurate and robust model will be created.

7.0 Results and Discussion

The evaluation results of the CNN model on the CIFAR-10 test set and discusses what these numbers mean in practice.

7.1 Overall Test Performance

On the test set, the CNN reaches an accuracy of 69.21% with a test loss of 0.94. In other words, the model correctly classifies around seven out of ten unseen images, which is a reasonable baseline for a relatively simple CNN trained without heavy tuning or regularisation. The

classification report shows a macro F1-score of 0.69 and a weighted F1-score of 0.69, indicating that performance is fairly consistent across classes and not dominated by only a few “easy” categories.

7.2 Class-wise Results

Class-wise metrics reveal which categories the model handles well and which remain challenging. For several classes, such as those with clearer shapes and textures, precision and recall are relatively high and the F1-score is around 0.73–0.82. This suggests that the CNN can reliably recognise objects with distinctive visual patterns. Some classes show noticeably lower scores. For example, one class reaches a recall of only 0.37, even though its precision is higher. This means the model is conservative for that class: when it predicts the label it is often correct, but it also misses many true examples and assigns them to other, visually similar classes.

7.3 Error Patterns

The error patterns are not random. Animal classes are most often misclassified as other animals, while vehicles tend to be confused with other vehicle classes. this is expected, because similar objects share edges, colours, and textures, which makes them harder to separate for a shallow CNN. From a practical perspective, the model is more reliable at distinguishing broad groups (for example, “animal” vs “vehicle”) than at making fine-grained decisions inside each group. For applications where this level of granularity is sufficient, the current model can already be used; for tasks that demand very precise class boundaries, further improvements are needed.

7.4 Interpretation and Implications

The results show that the CNN has successfully learned meaningful visual features from CIFAR-10 and delivers a solid baseline performance at around 70% accuracy. The relatively balanced macro and weighted F1-scores confirm that the model does not rely only on a subset of classes, even though some categories remain noticeably harder to classify.

The observed error patterns also provide a clear roadmap for future work. Techniques such as data augmentation, stronger regularisation (for example, dropout or batch normalization), or deeper CNN architectures could help the model better separate visually similar classes and push the accuracy beyond the current plateau. So, the model is a good starting point: it works reasonably well on unseen data, and the analysis in this section highlights concrete directions for systematic improvement in the next iteration.

8.0 Reflection

Implementing the BI dashboard and its data architecture has been a very concrete lesson in connecting technical design with real decision making. Working from raw source systems through ETL into a star-schema warehouse showed me that clean, consistent definitions of KPIs matter as much as any specific tool. Dealing with data quality issues, clarifying the grain of each fact table, and standardizing business terms sometimes felt tedious, but it directly reduced confusion for users and made the dashboard feel reliable.

The project made me learned how to build data systems for people, not just for correctness. Early versions of the dashboard were technically sound, yet user feedback revealed which visuals were confusing, which filters they really needed, and which questions were still unanswered. Iterating on that feedback, simplifying some views, and documenting key design choices made the solution easier to adopt and maintain. Overall, My group came away with a stronger appreciation for small, continuous improvements and for keeping the conversation open between data practitioners and stakeholders.